

Rack Audit

How It Works

A guided tour of the inspection app's Rack View — the scoped Reconciliation flow and the free-form Manual Mode — told through a real audit, AUD-0240.

Audience	Worked example	Version
Product, QA, new inspectors	AUD-0240 · Rack A-21	variation-3

1	What the app is for	3
2	The building blocks: audits, locations, pallets	4
3	Meet the example — AUD-0240	5
4	Normal Mode — reconciling the assigned scope	6
5	Manual Mode — reporting outside the scope	9
6	Design rationale — why it's built this way	11
7	Quick reference — colors & states	12

1 What the app is for

Rack Audit is a tablet app for warehouse inspectors. An admin schedules an **audit** — a task like "count every iPhone 15 Box across Rack A-21" — and assigns it to an inspector. The inspector walks the physical rack with the tablet, opens the app's **Rack View**, and works through the assigned pallets one at a time: scan what's really there, compare it against what the paperwork says should be there, and record the result.

The rest of the app supports that one core loop:

Dashboard

Today's ongoing audit, at a glance — what's assigned, how far along it is, and a one-tap way back into it.

Rack View

The main event — an interactive rack diagram plus the reconciliation form. Covered in depth in this document.

Audit Schedule

Every audit assigned to this inspector, past and upcoming, with status (Not Started / In Progress / Completed).

Reported Audits / Progress

A running log of everything scanned, matched, mismatched, or flagged — the paper trail an audit produces.

UX FRAMING

This is a **field tool**, not a back-office dashboard. Every design decision downstream assumes the person using it is standing in a warehouse aisle, tablet in one hand, possibly a torch or a damaged box in the other. Big tap targets, unambiguous color states, and as few decisions-per-pallet as possible.

2 The building blocks

Four concepts recur through the whole app. Getting these straight makes everything else in this document click.

- A Audit**
A scheduled task — e.g. "Full Count — Layouts A & B". Has a date range, a status, and (sometimes) a **target SKU**: a single item the audit exists to track down, rather than counting everything on the rack.
- B Location**
One physical slot on a rack — identified by rack, bay, level, and slot (e.g. Rack A-21, Bay B-03, Level 12, Slot 2 → shown to the inspector as `P-1202`). Each location has an **expected** SKU: what the warehouse's records say should be sitting there.
- C Pallet**
What's physically on a location — a SKU, a quantity, and a condition. The inspector's job is to find out what pallet is *actually* at a location and compare it to what's expected there.
- D Scan**
The one fact a barcode scan can tell you: **which SKU is on this pallet**. It cannot tell you the quantity or the condition — those are things the inspector has to look at and enter by hand.

UX FRAMING

That last point — a scan only proves identity, never quantity or condition — is the single decision that shapes the whole Reconciliation Form. It's why the app never lets a scan silently invent a quantity mismatch: nothing about qty/condition is "known" until the inspector deliberately looks and types it in.

3 Meet the example — AUD-0240

Every walkthrough below uses the same real audit from the app's demo data, so the two modes can be compared directly.

FIELD	VALUE
Audit	AUD-0240 — Full Count, Layouts A & B
Target SKU	SKU-1001 — "iPhone 15 Box"
Rack in focus	Rack A-21 — 4 physical bays (B-01 through B-04)
Assigned scope	38 locations across Bays B-01–B-03 are seeded with SKU-1001 — these are what the audit is <i>for</i>
Out of scope	Bay B-04, plus every other pallet on B-01–B-03 not carrying SKU-1001 — physically part of the rack, but not what this audit was assigned to check

That gap — *the rack has 4 bays, the audit only cares about 38 specific pallets across 3 of them* — is the exact situation both modes are built to handle:

- **Normal Mode** keeps the inspector focused on precisely those 38 pallets, and nothing else.
- **Manual Mode** is the escape hatch for when reality doesn't respect that boundary — e.g. the inspector notices a damaged pallet in Bay B-04, entirely outside the assignment, and needs to report it anyway.

4 NORMAL MODE Reconciling the assigned scope

4.1 — The canvas: only the assigned pallets are open

Rack View draws the whole physical rack — all 4 bays of A-21 — so the inspector always has full spatial context. But only the 38 pallets seeded with SKU-1001 are interactive:



Darker gray cells are the target pallets — tappable, and listed in the toolbar's Pallet dropdown. Everything else is dimmed and un-tappable: not hidden (the inspector can still see it's a pallet), just clearly not this audit's business.

EXAMPLE

Location `A-21-B-03-1202` (Bay B-03, Level 12, Slot 2 — shown as **P-1202**) is one of the 38. Tapping it, or picking "P-1202" from the Pallet dropdown, selects it on the canvas — the two stay in sync either direction.

4.2 — Selecting a pallet opens a split view

Tapping **Start Audit** splits the screen: the rack canvas stays visible on the left, and a Reconciliation Form opens on the right. This is deliberate — closing off the canvas would mean losing spatial context exactly when it's most useful (re-selecting a neighboring pallet, spotting where you are in the rack).

Reconciliation Form

SELECTED LOCATION DETAILS

Location Code	A-21-B-03-1202
Rack	A-21
Bay	B-03
Pallet	P-1202

Tap to Scan SKU

Scans the SKU code on the pallet, then checks it against what's expected here.

Notice what's *not* shown yet: the expected SKU. That's deliberate too — see 4.3.

4.3 — Why the expected SKU stays hidden until after scanning

UX FRAMING

If the expected SKU were visible before scanning, an inspector under time pressure could just scan *anything* that matches what they already see on screen, defeating the point of an audit. Hiding it until a real scan has happened removes that shortcut entirely — the comparison can only exist once a genuine scan is on record.

4.4 — After scanning: Expected vs. Scanned

Once the inspector scans the pallet's code, the dotted target is replaced by a side-by-side comparison and a status badge:

Reconciliation Form

SELECTED LOCATION DETAILS

Location Code

A-21-B-03-1202

Pallet

P-1202

Matched

EXPECTED

SKU-1001

iPhone 15 Box

Qty 12

SCANNED

SKU-1001

iPhone 15 Box

Qty 12 · Good

Enter actual quantity & damage found →

Cancel

Empty

Save & Scan Next

The badge here answers exactly one question: **is this the right SKU?** Quantity and damage are separate, secondary concerns — see the next section for why.

4.5 — The four outcomes

STATUS	COLOR	HOW IT HAPPENS
Matched		Scanned SKU = expected SKU, and (once the inspector fills it in) quantity and condition also check out.
Mismatch		The scanned SKU is <i>not</i> what's expected here — the wrong pallet was placed at this location. Nothing further to fill in; the inspector taps Raise Issue directly.
Quantity / Damage Mismatch		The right SKU is here, but after the inspector taps in and enters the real count or condition, it doesn't match the expected quantity or the item is damaged. Shown as a smaller badge <i>next</i> to "Matched" — the SKU identity is still correct, this is a secondary note.
Missing / Empty		The inspector physically finds no scanner code at this location at all — nothing to scan. Tapping Empty marks the location "missing" (dashed, dark gray) and lets them move on without a scan.

EXAMPLE — WALKING BAY B-03 IN ORDER

- P-0802 gets scanned and matches perfectly → green.
- P-1002 scans a wrong pallet's SKU entirely → red Mismatch, Raise Issue offered immediately.
- P-1201 — a code exists, but the physical box turns out empty when the inspector opens it — turns out to hold 10 units, not the expected 12 → Matched (green) badge plus a small amber "Quantity Mismatch" badge.
- P-0303 has no scanner sticker on it at all → inspector taps Empty → dashed dark-gray cell, counted among the 38 without blocking progress.

4.6 — Raising an issue, and the red dot

Whenever an inspector taps **Raise Issue** — for a Mismatch, or for a quantity/damage discrepancy — that pallet's canvas cell gets a small red dot in its corner. It stays there even after the panel closes or a different pallet is selected, so a walked-past problem is never silently lost from view.

4.7 — Moving through the rack

Save & Scan Next saves the current pallet's record and automatically selects the next one in sequence — the canvas highlight follows along, so the inspector never has to stop and manually re-tap the next cell.

5 MANUAL MODE Reporting outside the scope

THE SCENARIO THAT MOTIVATES IT

The inspector is walking Rack A-21 for AUD-0240 — scoped to 38 SKU-1001 pallets in Bays B-01–B-03. While passing **Bay B-04** — physically part of the same rack, but not assigned to this audit at all — they notice a pallet with a visibly damaged, leaking box. Left alone, it could contaminate or damage neighboring pallets. There's no way to report this through the normal flow, because Bay B-04 isn't even selectable.

5.1 — The toggle

Right next to the Pallet dropdown in the toolbar sits a **Manual Mode** switch. Flipping it on:

- Opens up **every pallet in the physical rack** for selection — not just the 38 assigned ones.
- Shows an amber warning banner confirming the mode is active, so it's never ambiguous which mode the inspector is in.
- Leaves the split view exactly as it was — toggling doesn't collapse the canvas back to full-screen or lose the current selection unnecessarily.



In-scope pallets keep the exact same look they always had. Newly-opened, out-of-scope pallets get a distinct **amber dashed border** instead — visually flagged as "only reportable because Manual Mode is on", not blended into the normal audit.

5.2 — Still scan first

UX FRAMING

It would be tempting to let Manual Mode skip straight to a free-text report — but that would make it too easy to log an issue against the wrong pallet, or one nobody actually checked. Manual Mode keeps the same discipline as Normal Mode: **identify by scan first**. The only thing that changes is there's no "expected SKU" to compare the scan against — it's simply revealed as-is.

Manual Issue Report

SELECTED LOCATION DETAILS

Location Code	A-21-B-04-0301
Rack	A-21
Bay	B-04

Tap to Scan SKU

Scans the SKU code on this pallet so you can report what you actually found here.

5.3 — Reporting the problem

Once scanned, the form shows what was found and lets the inspector fill in the rest — quantity, a **Damage** picklist (Good / Broken / Wet / Open Package / Other), and full evidence: photo, audio note, video, or a written note.

Manual Issue Report

SKU

SKU-3301 — Plastic Crate Blue

Quantity

3

Damage

• Broken

1 photo attached · note added

Cancel

Raise Issue

5.4 — Confirmed, not silently advanced

UX FRAMING

An earlier version of this flow auto-advanced to the next pallet the instant "Raise Issue" was tapped — which looked broken, because the form just reset to another blank-looking pallet with no visible confirmation. The fix: **Raise Issue now stays put** and flips into a confirmed "Issue Raised ✓" state. Moving to a different pallet is a separate, deliberate tap on the canvas — appropriate for a mode whose whole premise is "wherever I happen to spot a problem next," not a fixed sequence.

5.5 — Reviewing a report later

Re-selecting an already-reported pallet doesn't reopen the full form — it shows a compact, tappable summary instead:

Issue Raised

SKU-3301

Plastic Crate Blue

Qty 3 · Broken

Tap to see details

Tapping it expands back into the full, editable form for correction if needed.

5.6 — Turning Manual Mode back off

If the inspector is on an out-of-scope pallet when they flip the switch off, the app doesn't just strand them on an invalid selection. It picks up where the normal audit sequence would have continued: **the next expected pallet** after that position is auto-selected, so they can drop straight back into Normal Mode's flow.

6 Design rationale

A short summary of the principles that came up repeatedly across both modes:

- 1 A scan proves identity, nothing else**

Quantity and condition are always separate, deliberate human inputs — never inferred from a scan. This is why the expected SKU stays hidden pre-scan, and why a fresh scan never shows a false quantity mismatch.
- 2 Scope is visible, not just enforced**

Out-of-scope pallets aren't hidden — they're faded (Normal Mode) or dashed (Manual Mode), so the inspector always understands the boundary rather than just hitting an invisible wall.
- 3 Every action gets visible proof**

Raising an issue shows a badge, a red dot on the canvas, and (in Manual Mode) a confirmed state on the button itself — never a silent state change the inspector has to trust happened.
- 4 Canvas and form stay together**

The split view means spatial context (where am I in the rack) is never sacrificed for form focus — critical when the whole point of the app is physically walking a real rack.
- 5 Selection is unmistakable**

The currently selected pallet blinks light blue with a dark blue border — but only when unresolved. Once a pallet has a real status (matched, mismatch, missing), re-selecting it keeps that true color, so a resolved pallet never looks like it reverted to "not yet checked".

7 Quick reference

Canvas cell states

STATE	LOOK	MEANING
Not yet checked (in scope)		Expected pallet, not yet selected or resolved this session.
Out of scope		Normal Mode — faded, not tappable.
Out of scope (Manual Mode)		Now selectable — dashed amber border marks it as Manual-Mode-only.
Selected, unresolved		Light blue fill, dark blue border, blinking.
Selected, already resolved		Keeps its true status color; only the border turns blue and blinks.
Matched		Right SKU, right qty, good condition.
Mismatch		Wrong SKU scanned at this location.
Issue (qty/damage)		Right SKU, but quantity or damage doesn't check out.
Missing / Empty		No scanner code physically found.

Mode comparison

	NORMAL MODE	MANUAL MODE
Selectable pallets	Only the assigned/expected ones	Every pallet in the physical rack
Comparison shown	Expected vs. Scanned	None — just reports what was scanned
Scan required?	Yes	Yes
Primary outcome	Matched / Mismatch / Missing	Issue Raised
Progression	Save & Scan Next (sequential)	Manual, pallet-by-pallet via canvas taps
Revisit an entry	Reloads the saved scan for editing	Collapses to "Tap to see details" summary